

Artificial Intelligence

Instructor: Kietikul Jearanaitanakij

Department of Computer Engineering

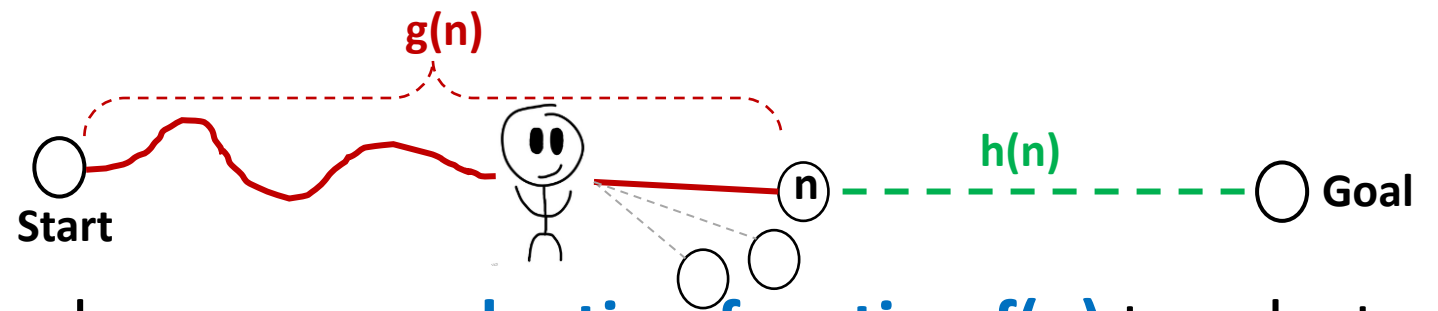
King Mongkut's Institute of Technology Ladkrabang

Lecture 4

Heuristic & Adversarial Search

- Heuristic Search Strategies
- Optimal Decisions in Games
- Alpha-Beta Pruning

Heuristic Search

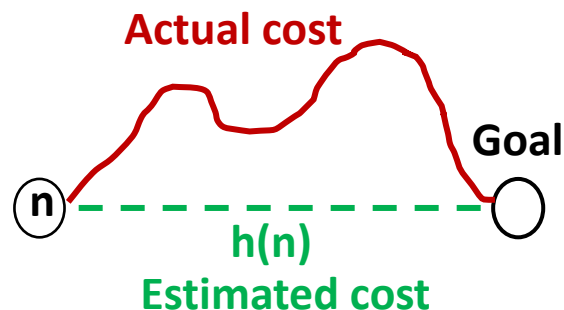


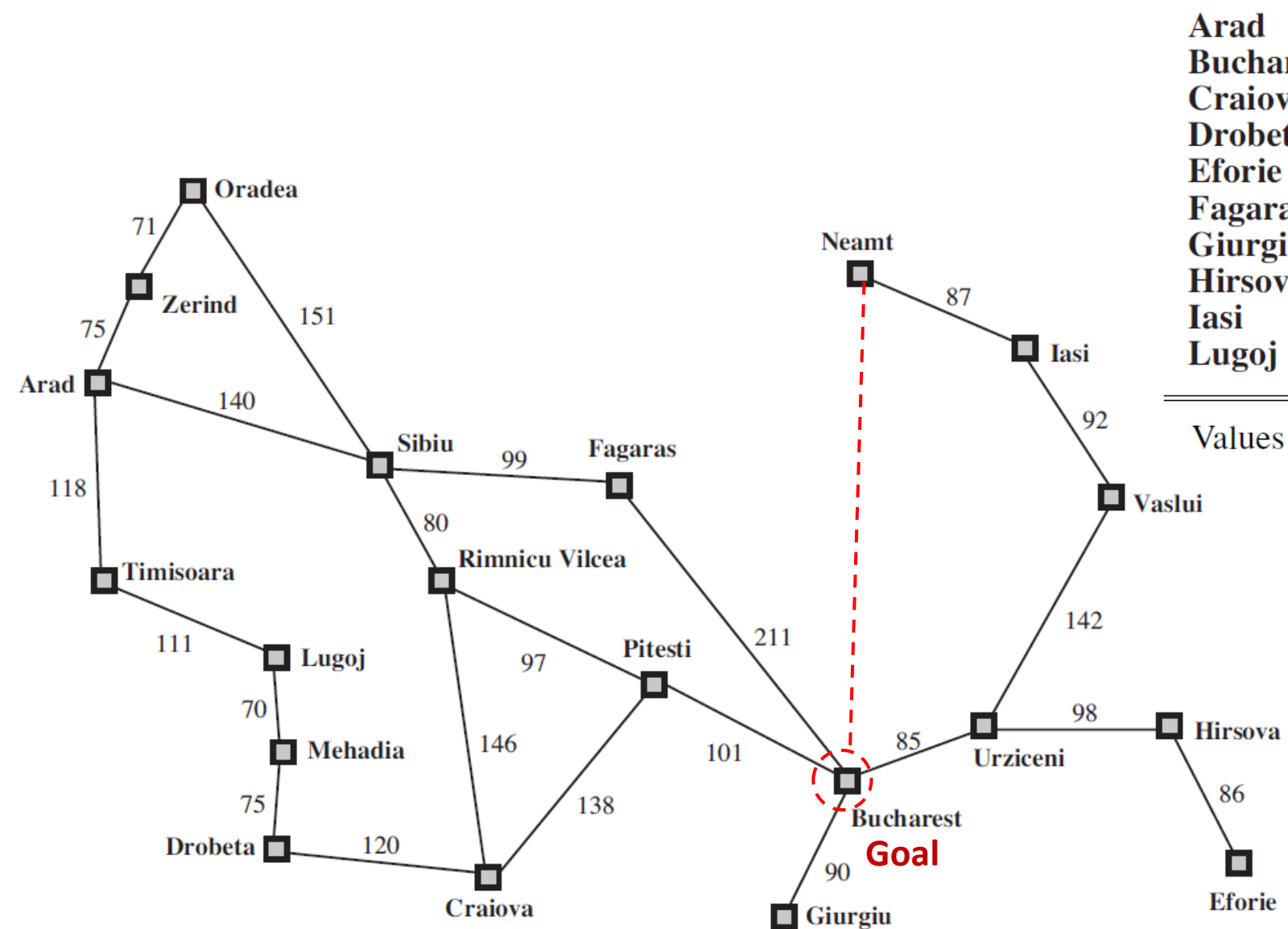
- The traditional heuristic search uses an **evaluation function $f(n)$** to select a node to be expanded.
- **Evaluation function $f(n) = \text{Heuristic function } h(n) + \text{Actual Path Cost } g(n)$**
- **Heuristic function $h(n)$** provides a clue to guess which neighbor of a node will lead to a goal.
 - $h(n)$** = **estimated** cost of the **cheapest path** from the state at **node n** to a **goal state**.
Where $h(n)$ is nonnegative and $h(n) = 0$ if n is a goal state.
- The heuristic function is constructed as a cost estimate, so the node with the **lowest heuristic value is expanded first**.
- Heuristic search selects a node to be expanded by the uniform-cost search.

Heuristic Search Strategies

1. Greedy best-first search : $f(n) = h(n) + \cancel{g(n)}$

- Expand the node that tends to be closest to the goal.
- It evaluates nodes by using just the heuristic function, i.e., $f(n) = h(n)$.
- In the route-finding problems in Romania, we can use the **straight-line distance heuristic**, h_{SLD} .
- Good heuristic function never overestimates the actual cost.

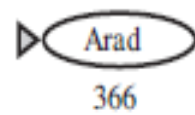




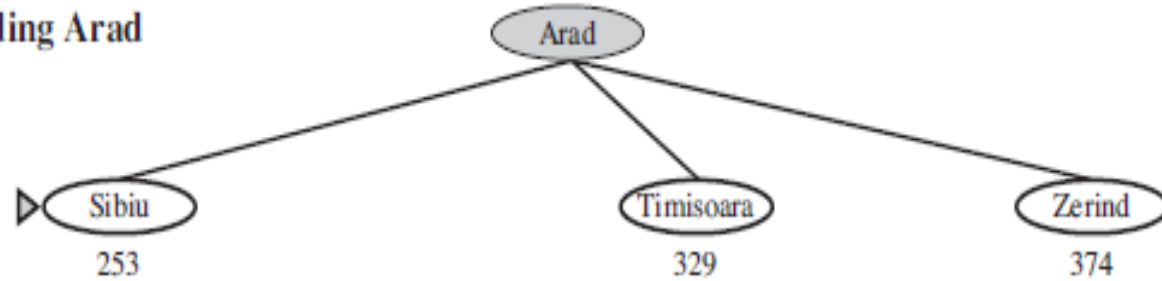
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

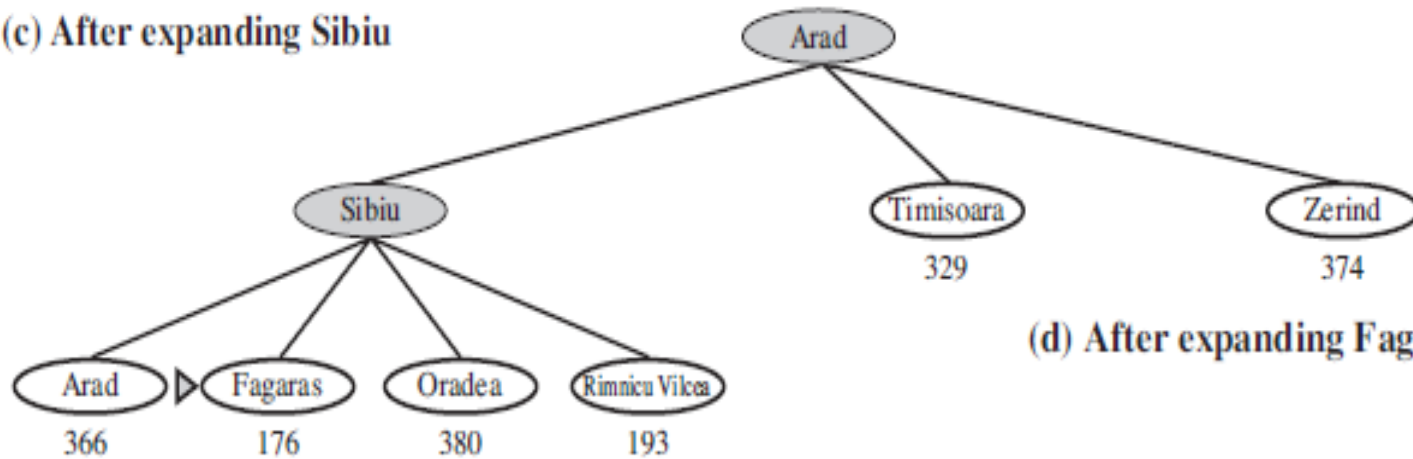
(a) The initial state



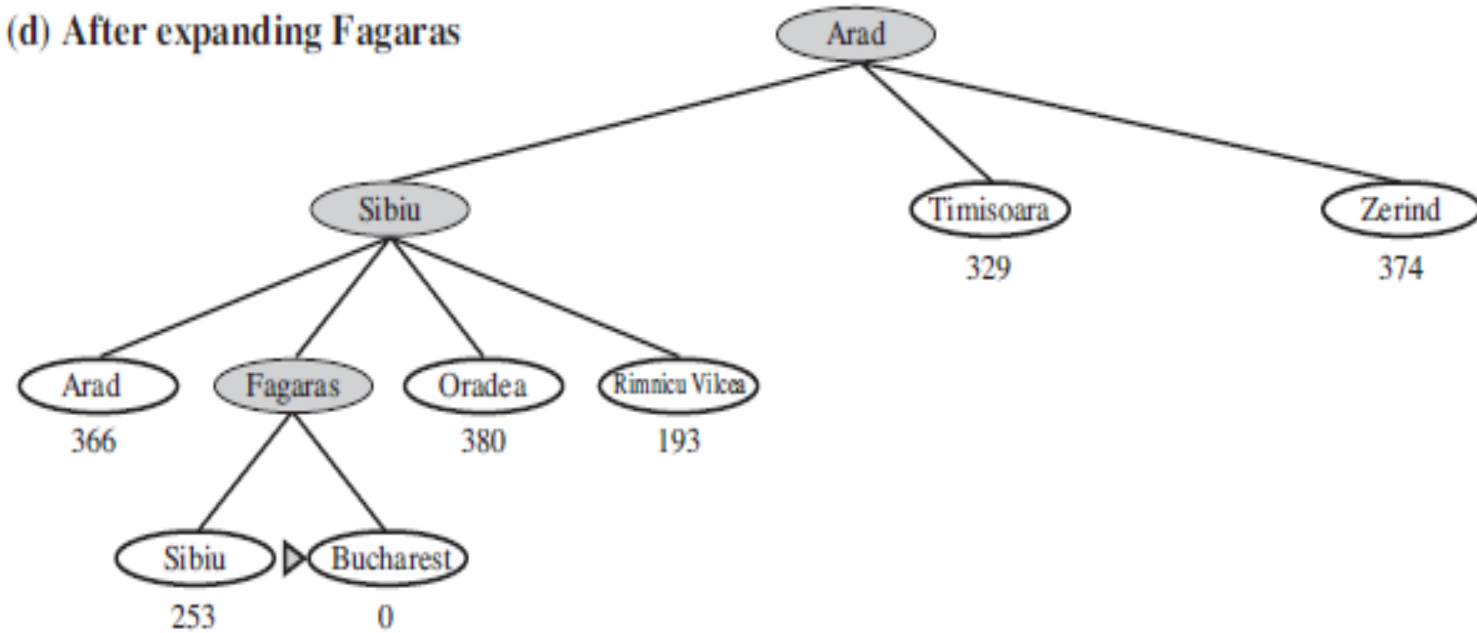
(b) After expanding Arad



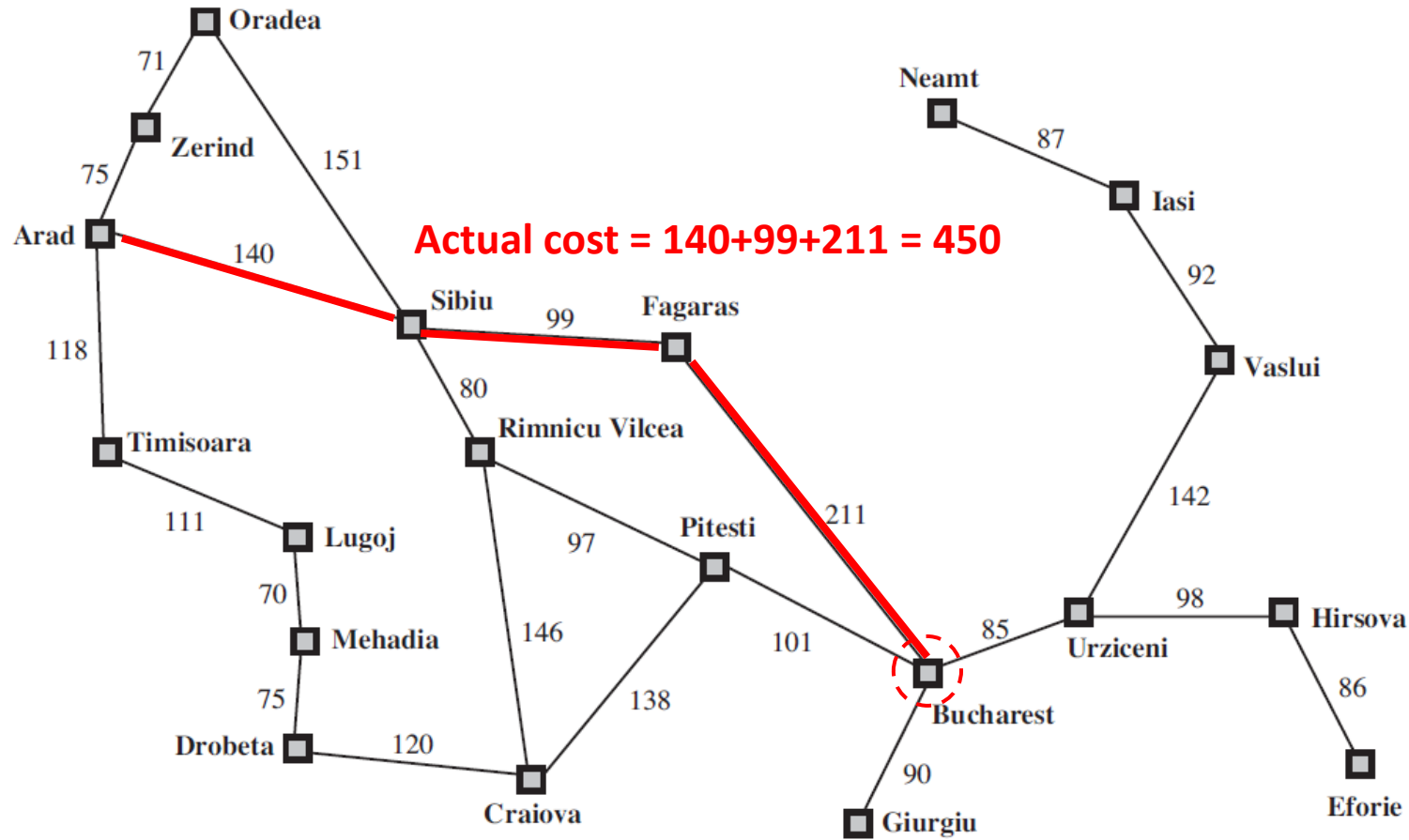
(c) After expanding Sibiu



(d) After expanding Fagaras



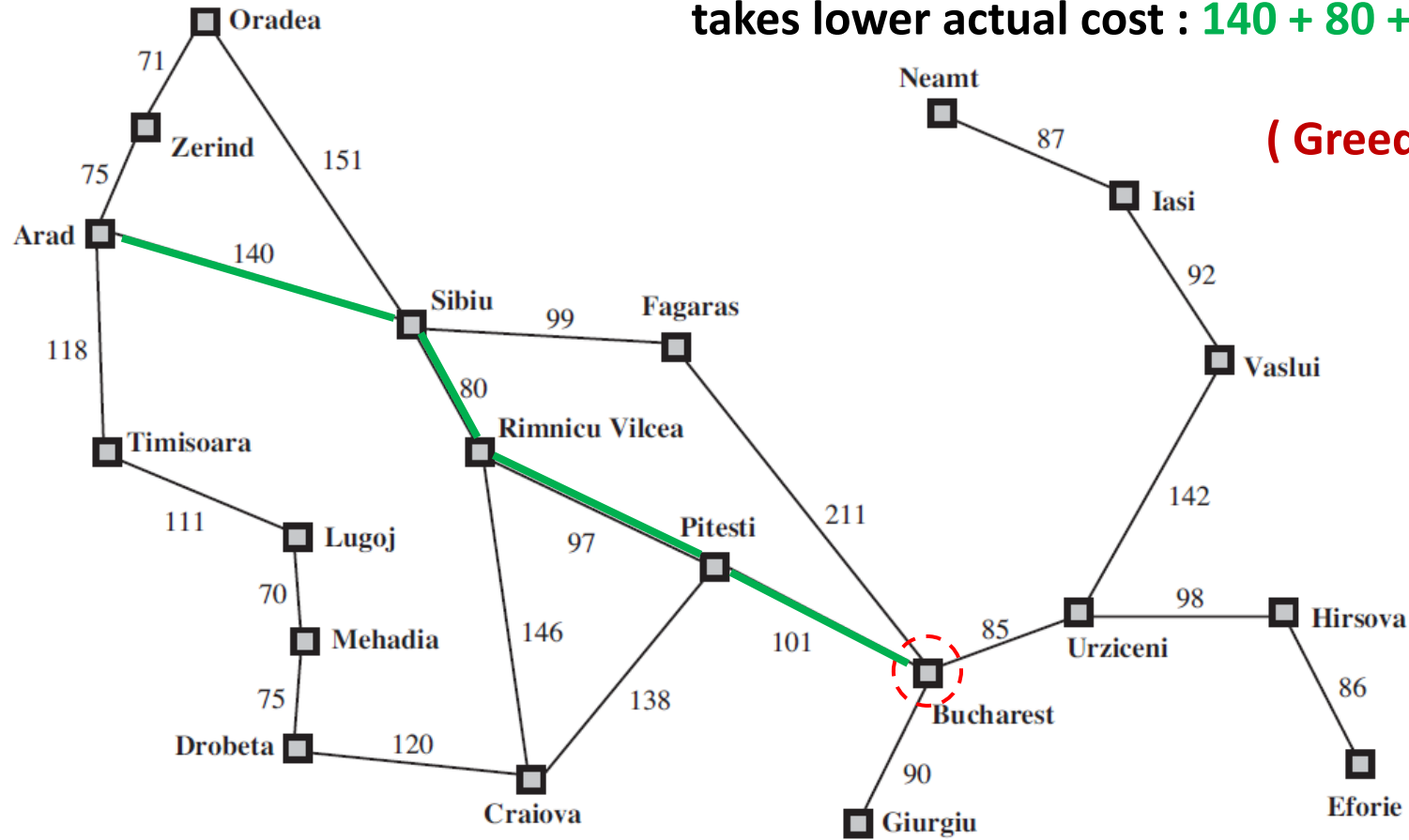
Searching from Arad to Bucharest by using greedy best-first search



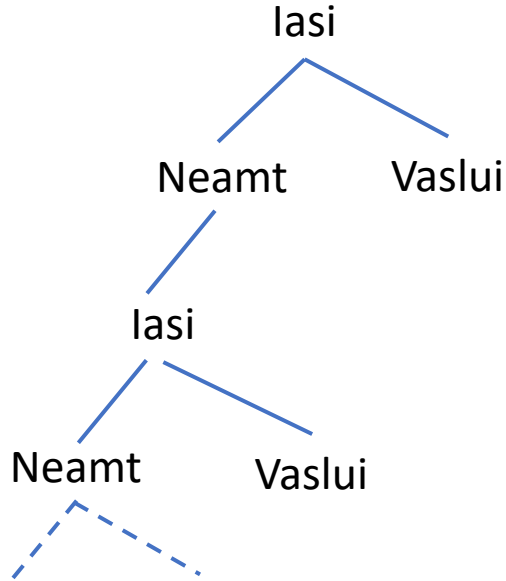
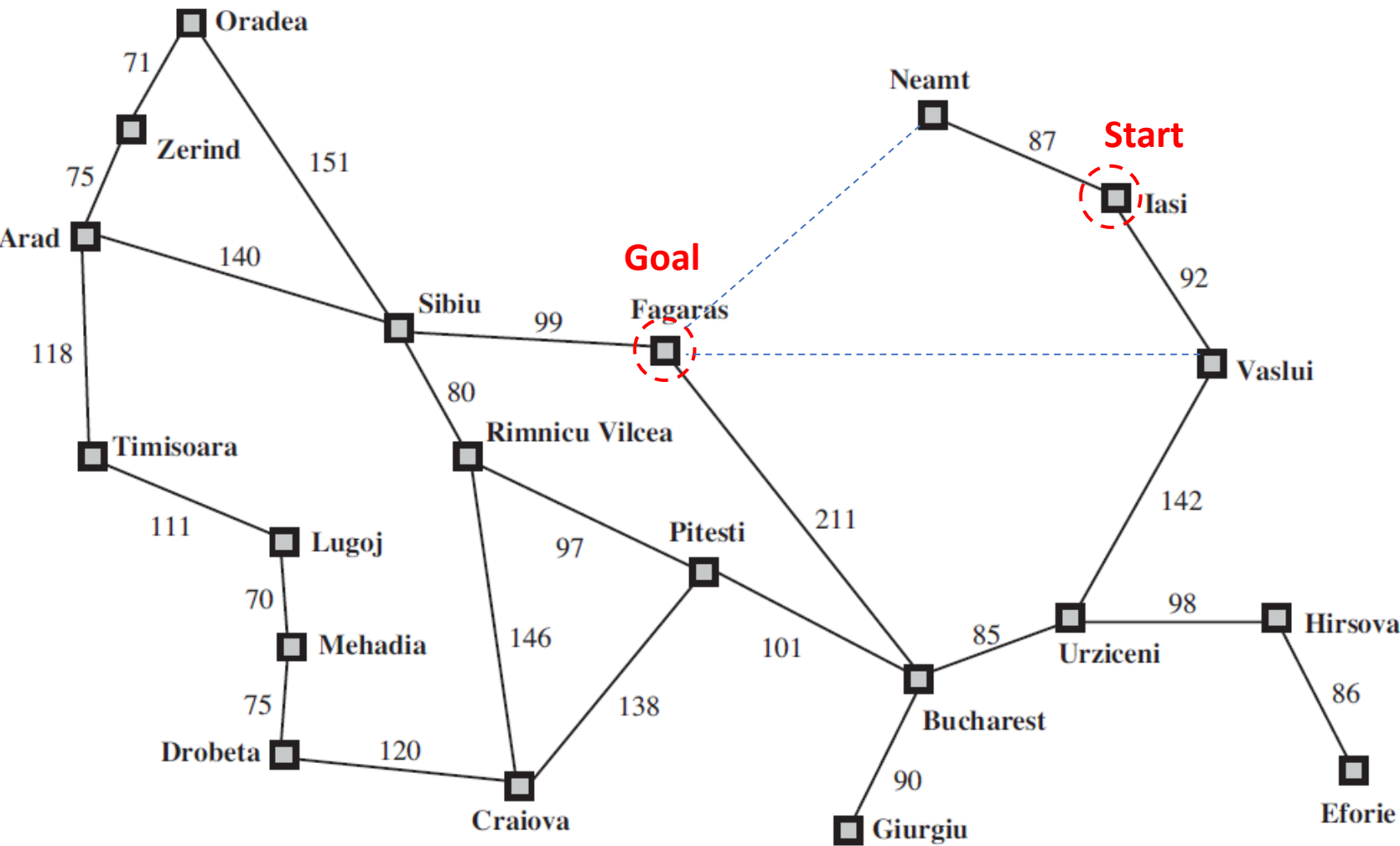
Searching from Arad to Bucharest by using greedy best-first search

However, **Arad -> Sibiu -> Rimnicu -> Pitesti -> Bucharest**
takes lower actual cost : $140 + 80 + 97 + 101 = 418$

(Greedy search is not optimal !)



Greedy best-first search is also **incomplete**.



Infinite loop

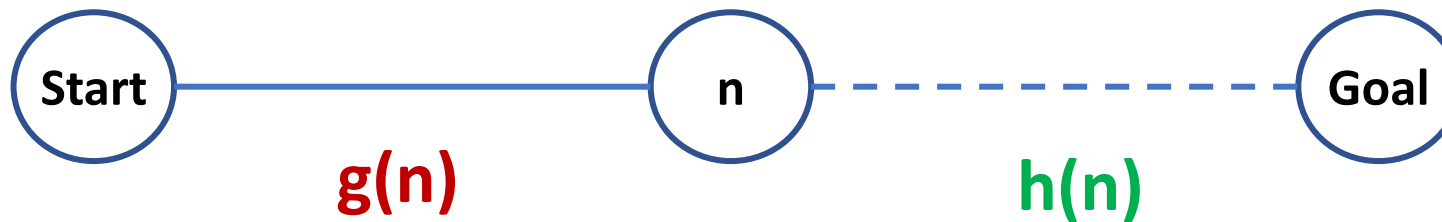
Time & Space Complexities = $O(b^m)$
Where m is the max depth of the search space.

Informed (Heuristic) Search Strategies

2. A* search (“A-star search”)

It evaluates nodes by combining $g(n)$, *the path cost to reach the node n* , and $h(n)$.

$$f(n) = h(n) + g(n)$$

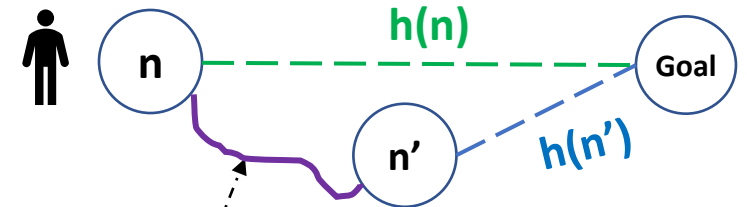


$f(n)$ = estimated cost of the cheapest solution through n .

Conditions for optimality of A* search

1. $h(n)$ must be an admissible heuristic.

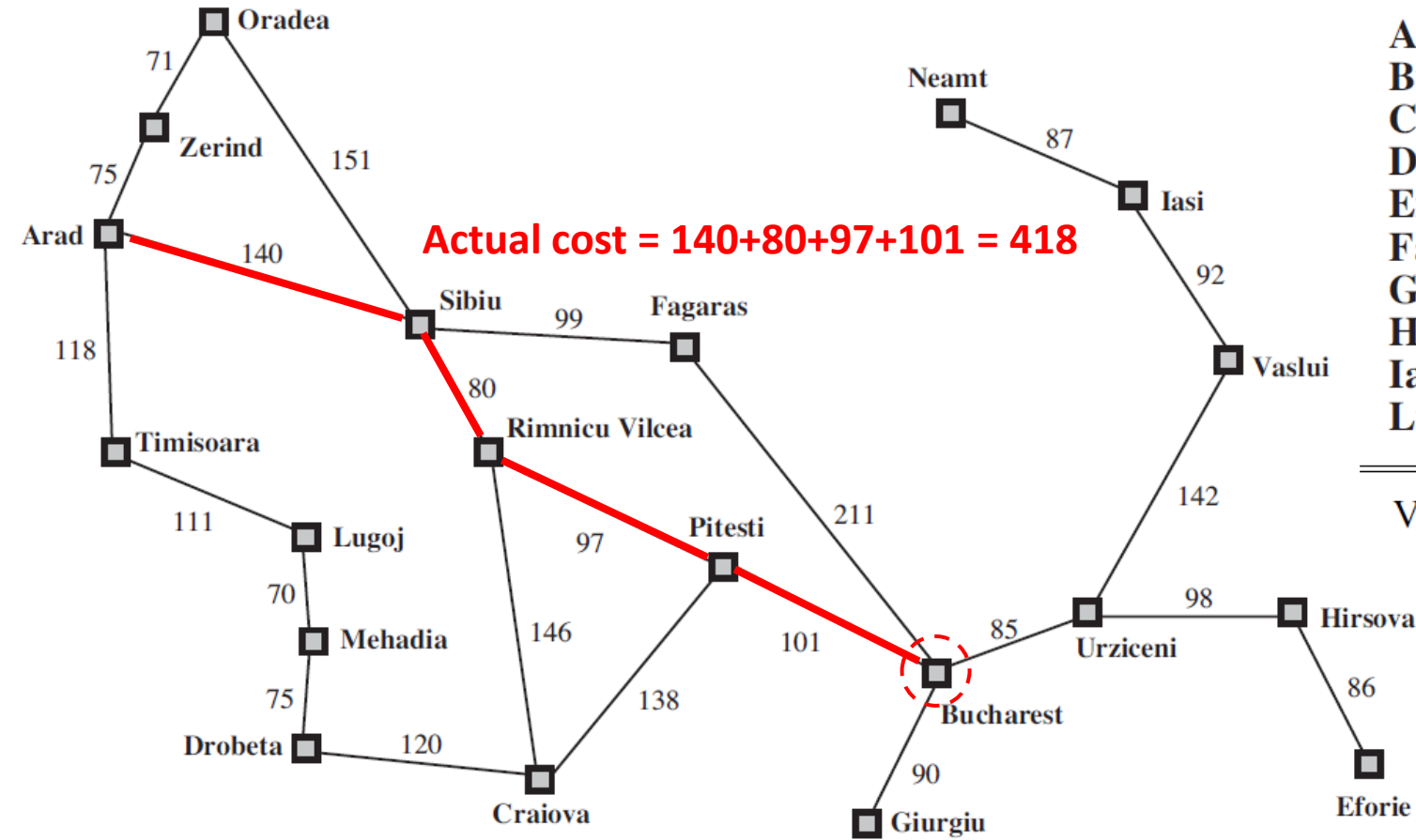
An admissible heuristic is one that **never overestimates the cost to reach the goal**. Ex: Straight-line distance is admissible because it is the shortest path between any two points.



2. $h(n)$ must be consistent (monotonicity). $h(n) \leq \text{step_cost}(\text{from } n \text{ to } n') + h(n')$

A heuristic $h(n)$ is consistent if, for every node n and every successor n' of n , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus $h(n')$.

Searching from Arad to Bucharest by using A* search

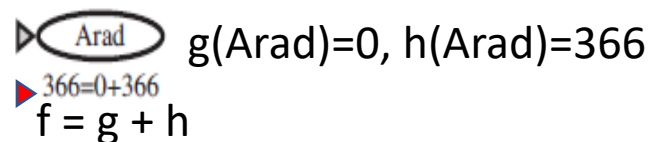


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

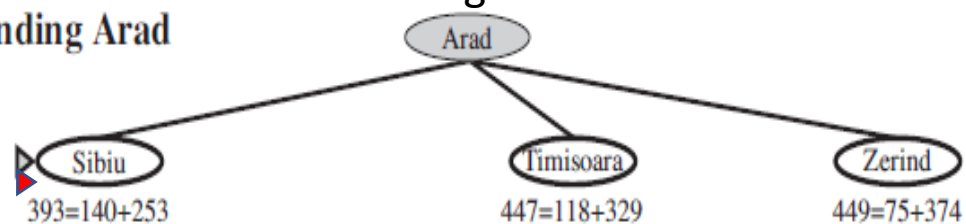
Values of h_{SLD} —straight-line distances to Bucharest.

A* search is optimal !

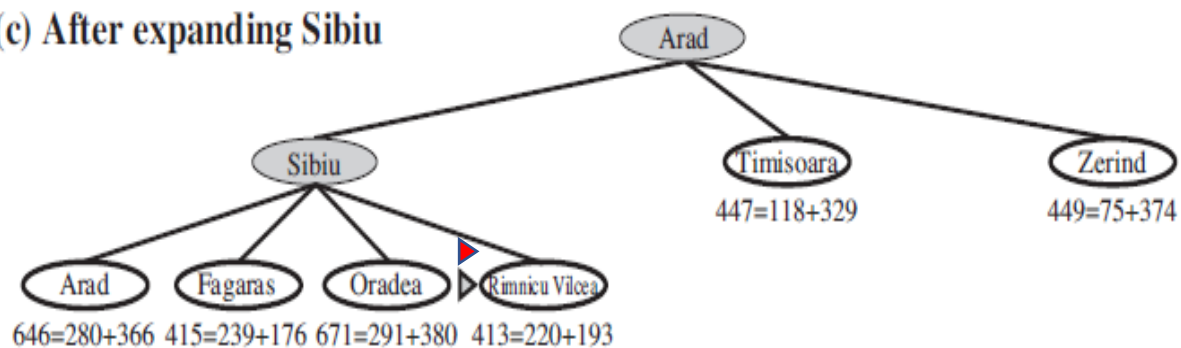
(a) The initial state



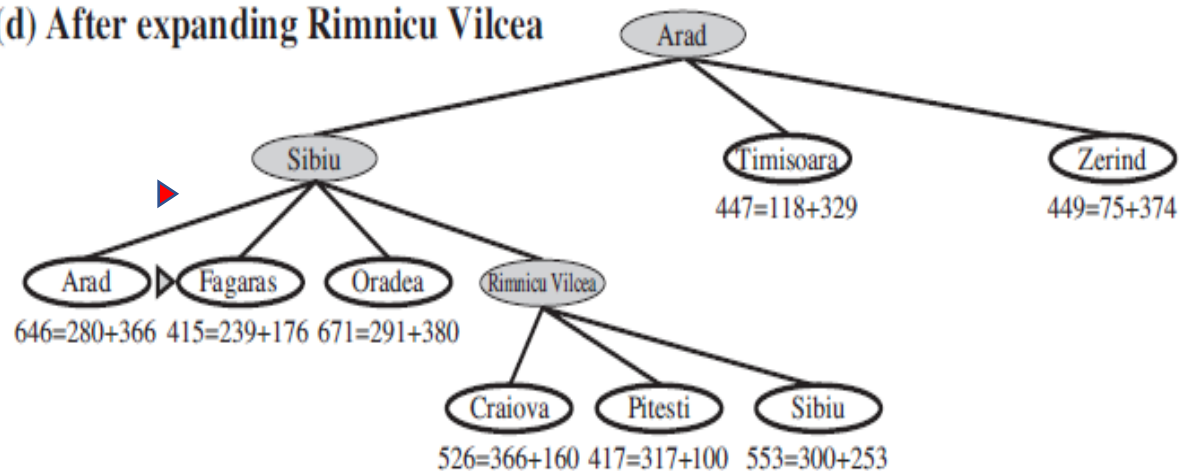
(b) After expanding Arad



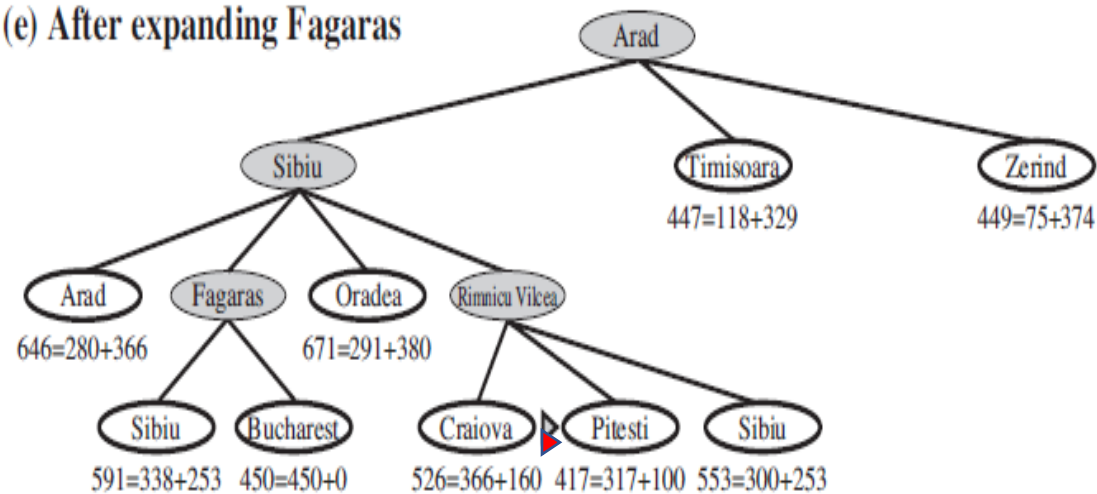
(c) After expanding Sibiu



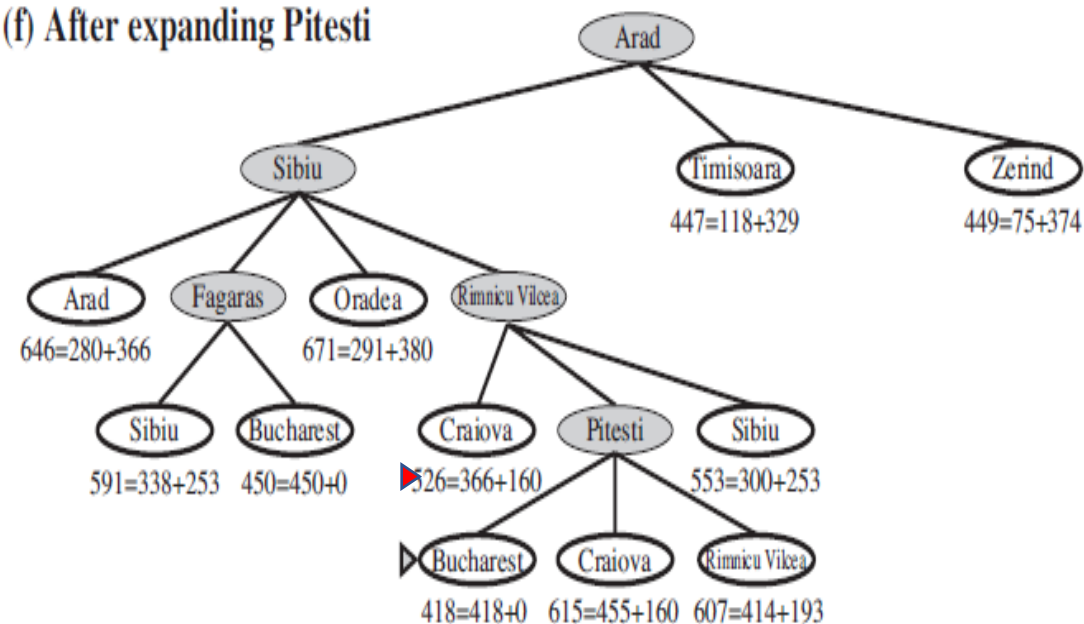
(d) After expanding Rimnicu Vilcea

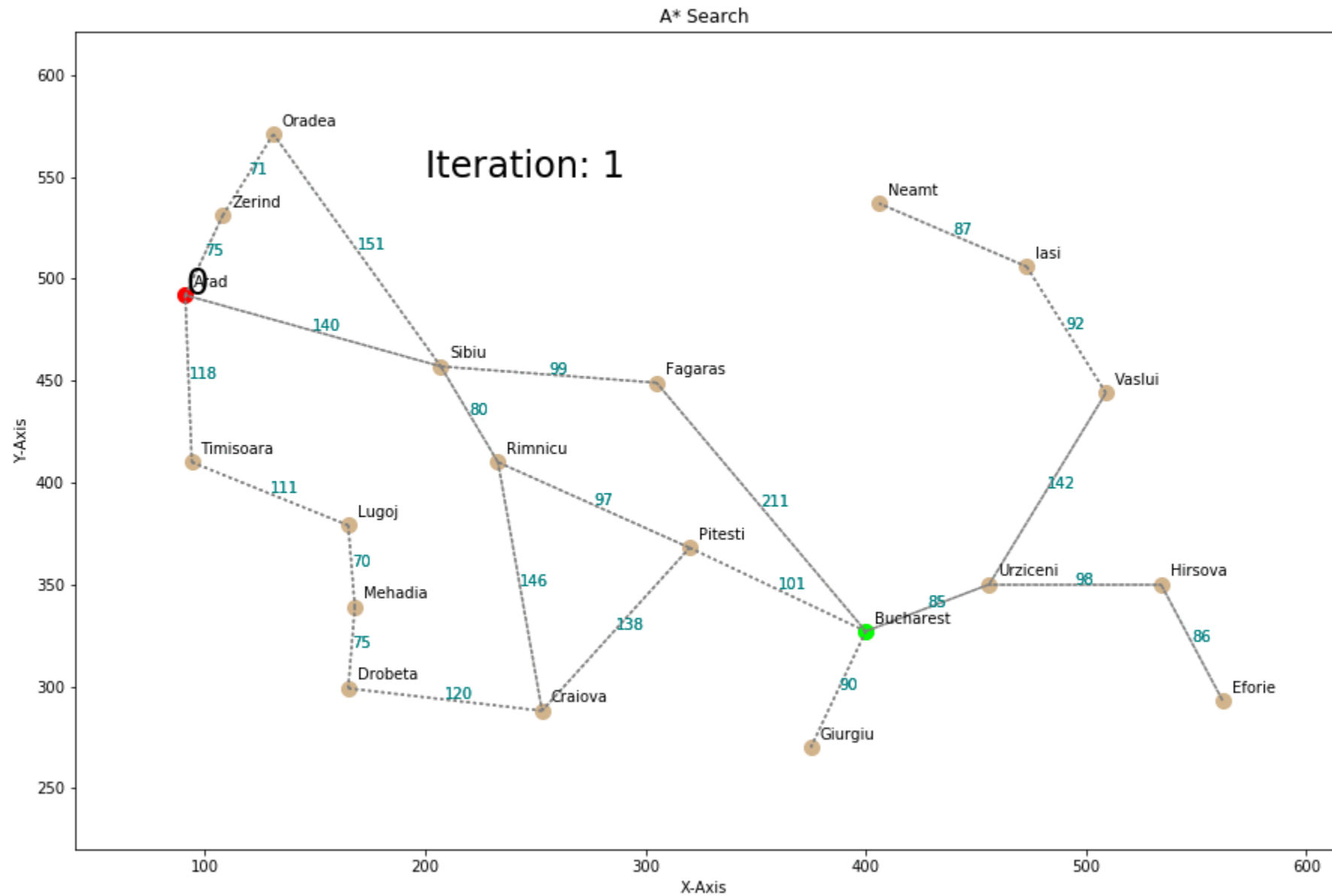


(e) After expanding Fagaras

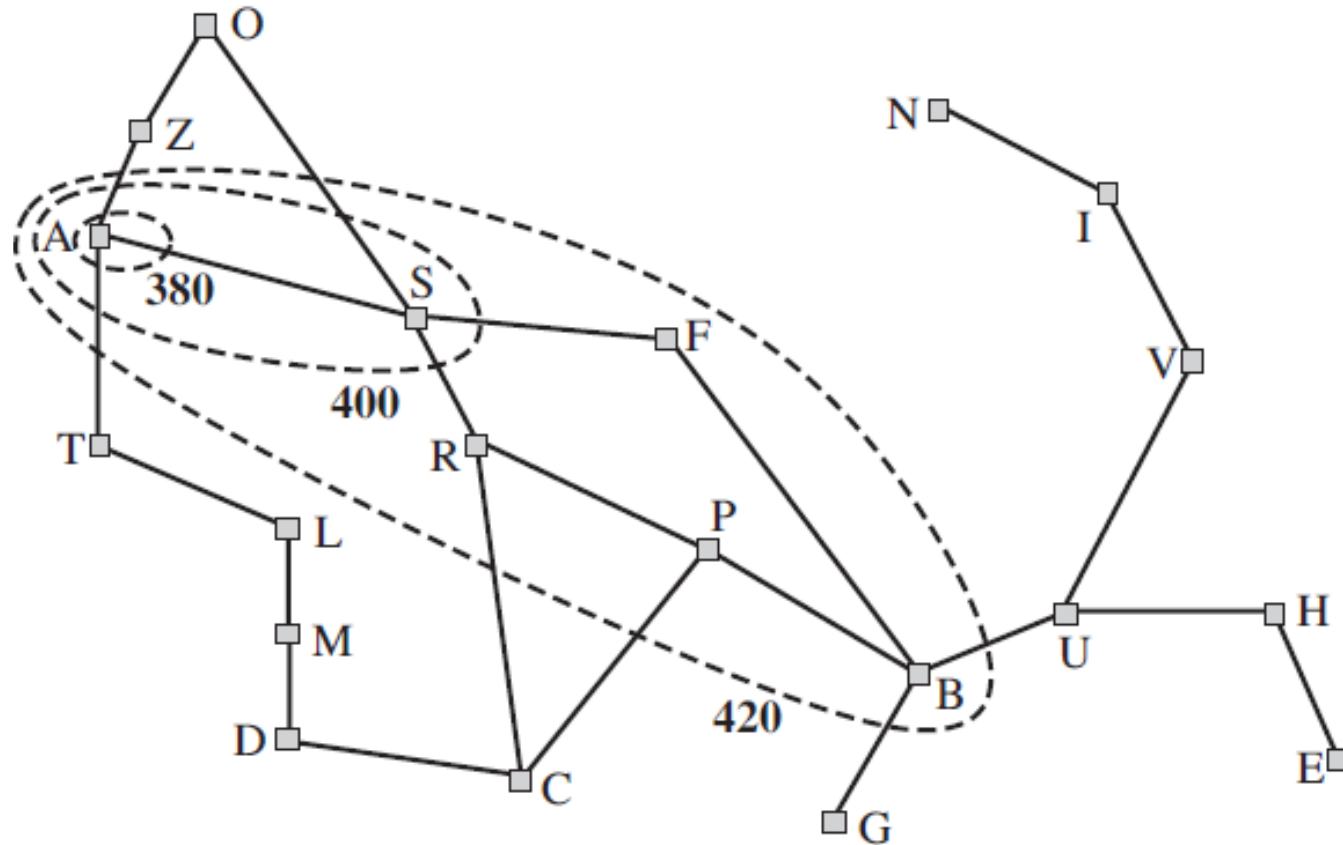


(f) After expanding Pitesti





Contour plots at $f = 380$, $f = 400$, and $f = 420$, with Arad as the start state. All contours have a similar direction toward the goal state (B). Hence, **A* search is complete.**



However, A* may not be suitable for large-scale problems since it has bad computation time and space complexity.

Examples of Heuristic Function

8-puzzle problem

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

A typical instance of the 8-puzzle. The solution is 26 steps long.

$$h1 = 8$$

$$h2 = 3 + 1 + 2 + 2 + 3 + 2 + 2 + 3 = 18$$

$h1$ = the number of misplaced tiles.

$h1$ is an admissible heuristic because it is clear that any tile that is out of place must be moved at least once.

$h2$ = the sum of the distances of the tiles from their goal positions.

Manhattan (city block) distance is the sum of the horizontal and vertical distances.

d	Search Cost (nodes generated)			Effective Branching Factor		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

Figure 3.29 Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A^* algorithms with h_1 , h_2 . Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d .

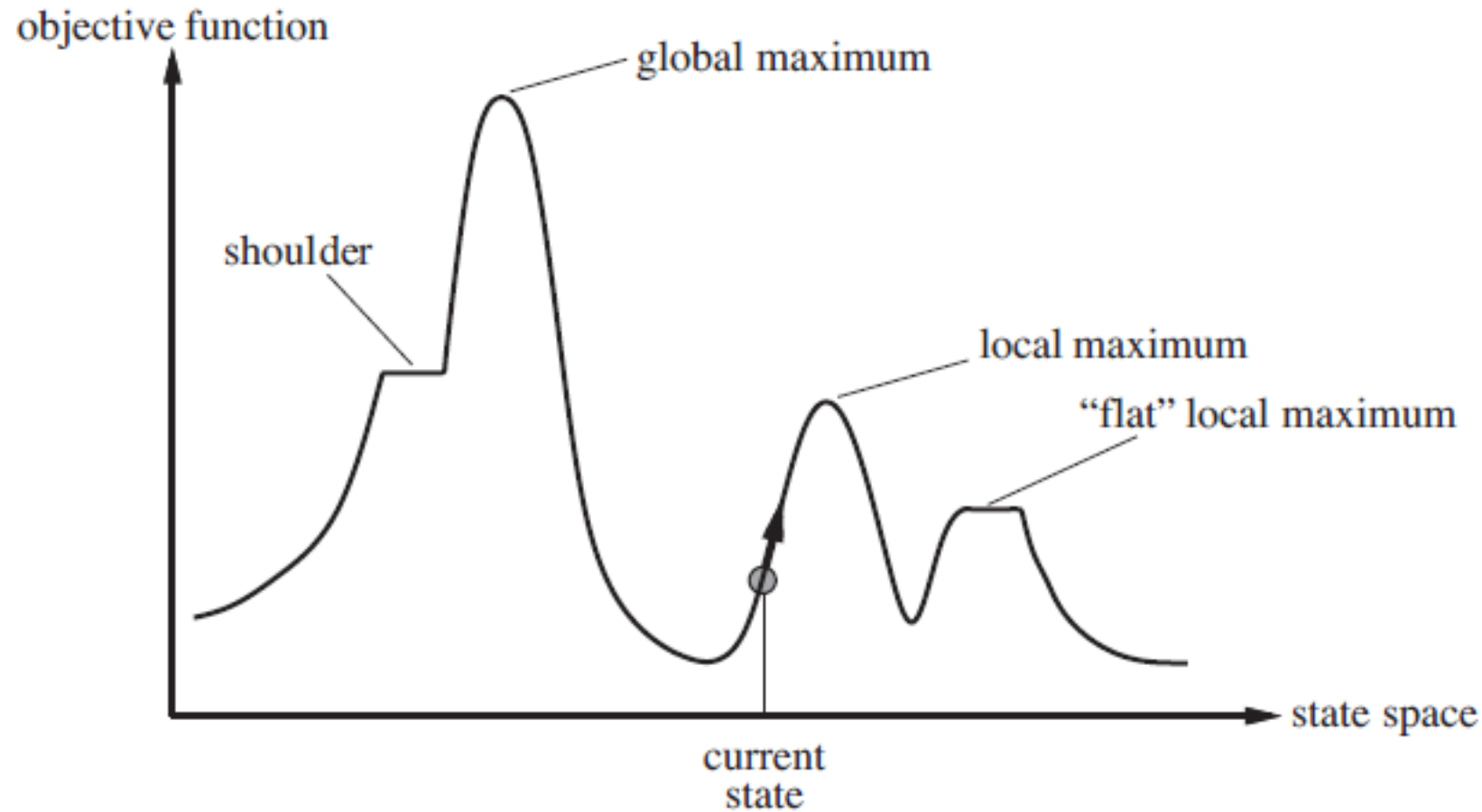
h_2 is better than h_1 , and is far better than using iterative deepening search.

Local Search Algorithms and Optimization Problems

- In many problems, however, **the path to the goal is irrelevant**. For example, in the 8-queens problem, what matters is the final configuration of queens, not the order in which they are added.
- Other examples are IC design, factory-floor layout, job scheduling, automatic programming, telecommunications network optimization, vehicle routing, and portfolio management.
- Local search algorithms operate using **a single current node** and **move to its neighbors**.

- Although local search algorithms are not systematic, they have two key advantages:
 1. They use **very little memory**—usually a constant amount; and
 2. They can often find **reasonable solutions** in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.
- Local search algorithms are useful for solving pure **optimization problems**, in which the aim is to find the best state according to an **objective function**.

State-space landscape



A one-dimensional state-space landscape. The aim is to find the global maximum.

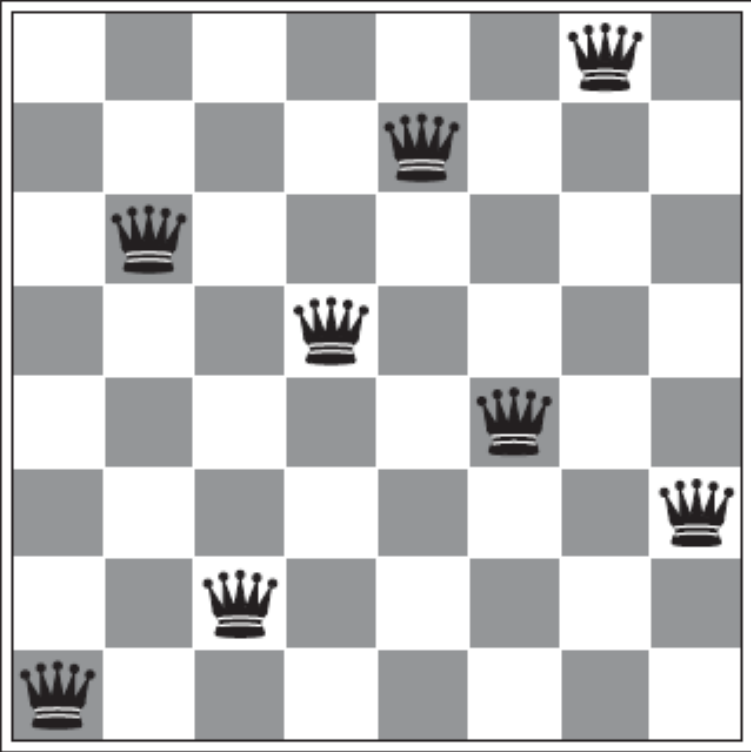
Local Search: Hill-climbing Search

```
function HILL-CLIMBING(problem) returns a state that is a local maximum  
    current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)  
    loop do  
        neighbor  $\leftarrow$  a highest-valued successor of current  
        if neighbor.VALUE  $\leq$  current.VALUE then return current.STATE  
        current  $\leftarrow$  neighbor
```

The hill-climbing search algorithm (steepest-ascent version) is simply a loop that continually moves in the direction of increasing value—that is, uphill. It terminates when it reaches a “peak” where no neighbor has a higher value.

The algorithm does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function.

Example of using hill-climbing search



$h = 1$

- **A state** formulation: each state has 8 queens on the board, one per column.
- The **successors** of a state are all possible states generated by moving a single queen to another square in the same column (so each state has $8 \times 7 = 56$ successors).
- **Heuristic cost function** h is the number of pairs of queens that are attacking each other.

Example of using hill-climbing search

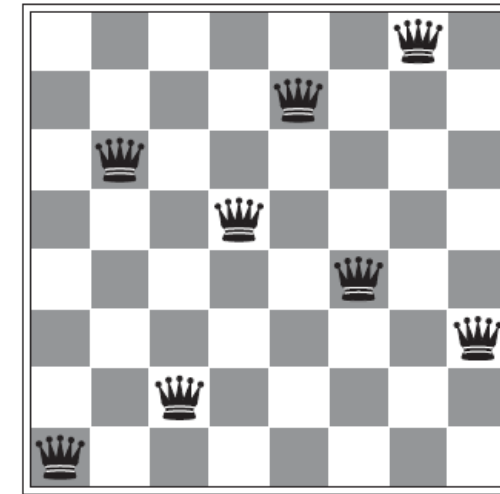
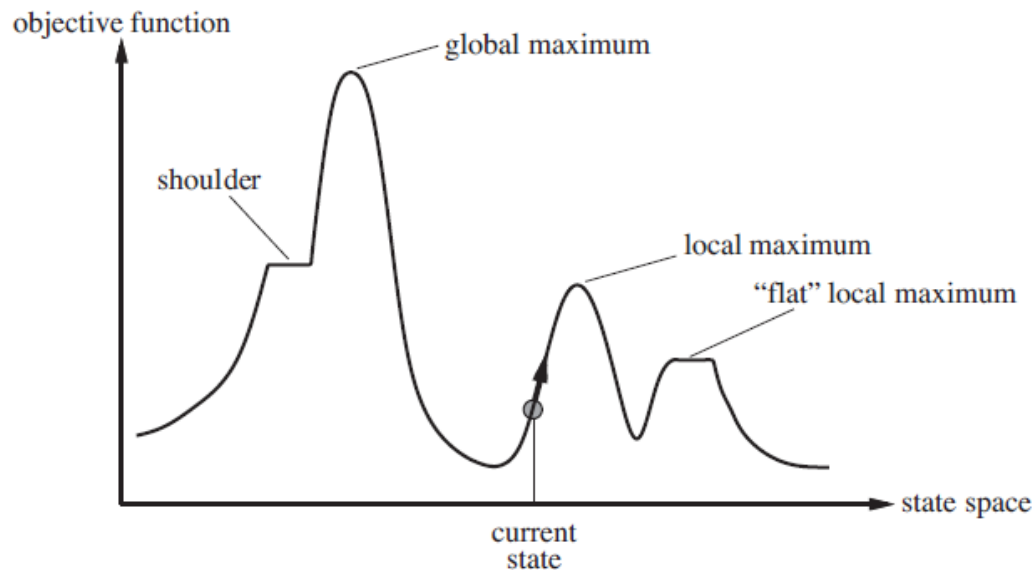
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	👑	13	16	13	16
👑	14	17	15	👑	14	16	16
17	👑	16	18	15	👑	15	👑
18	14	👑	15	15	14	👑	16
14	14	13	17	12	14	12	18

$h = 17$

- The figure shows the values of all its successors, with the best successors having $h=12$. (8 best successors which have $h=12$)
- Hill-climbing algorithms typically choose randomly among the set of **best successors** if there is more than one.
- Hill climbing is sometimes called **greedy local search** because it grabs a good neighbor state without thinking ahead about where to go next.

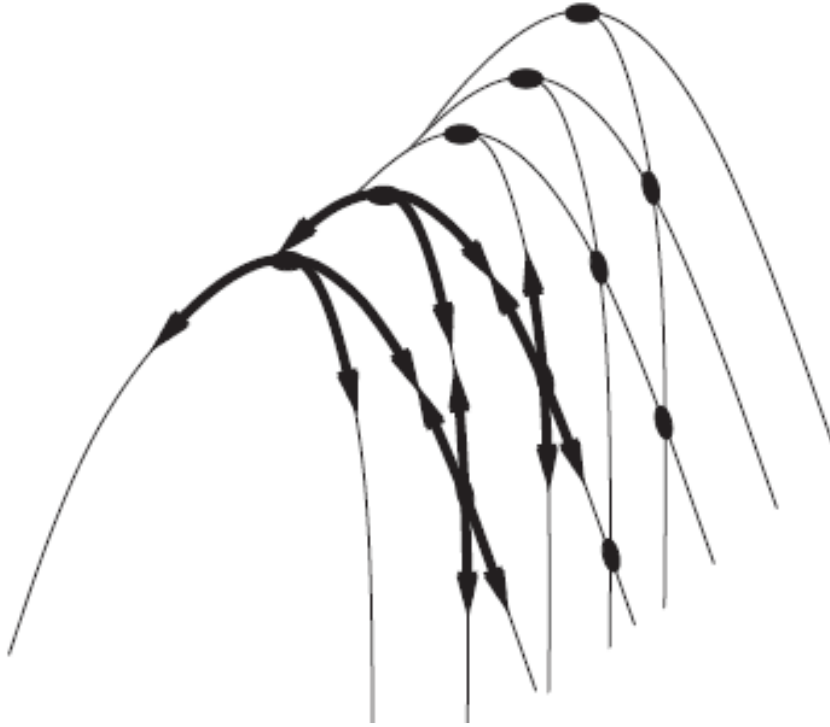
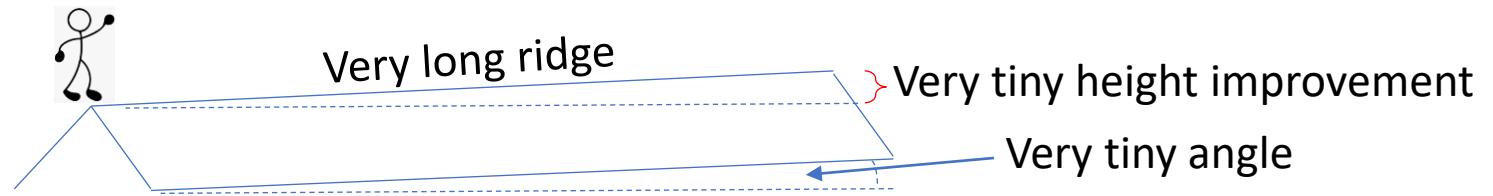
Unfortunately, hill climbing often gets stuck for the following reasons:

1. **Local maxima**: a local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum.



This state is a local maximum (i.e., a local minimum for cost h); every move of a single queen makes the situation worse.

2. Ridges:



Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.

3. **Plateaux**: A flat area of the state-space landscape. **An agent searches without direction back and forth.** It might get lost in the plateau forever.



Simulated Annealing Search

A hill-climbing algorithm never makes “downhill” moves. It tends to get stuck in a local maximum.

Simulated annealing: Annealing is the process of hardening metals (or glass) by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach the optimal state.



function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem
schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

$T \leftarrow$ *schedule*(t)

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

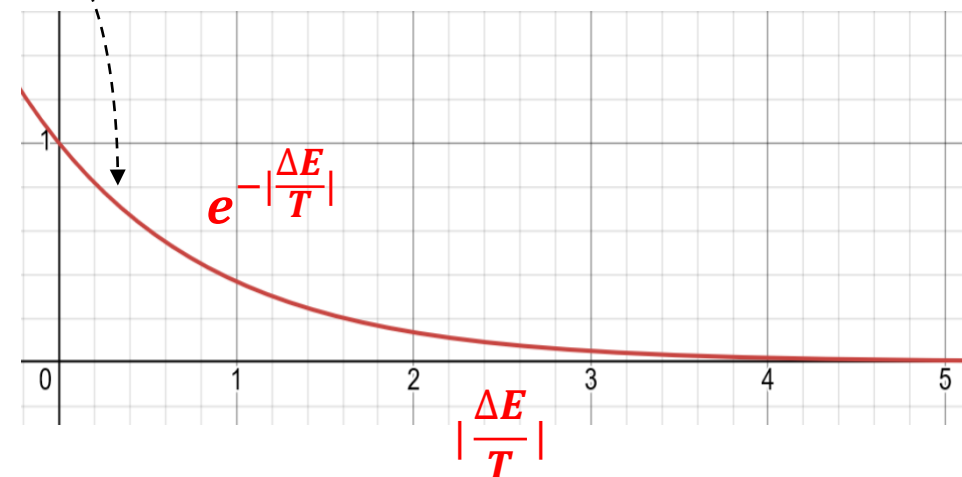
$\Delta E \leftarrow$ *next*.VALUE – *current*.VALUE

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

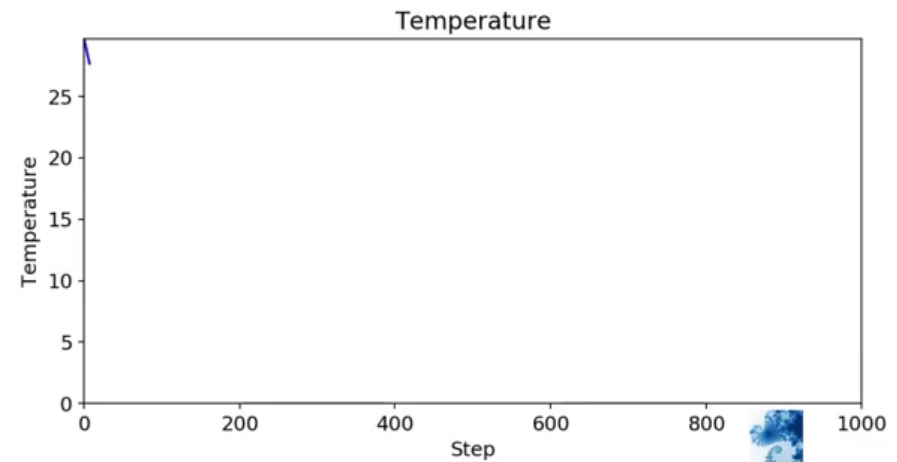
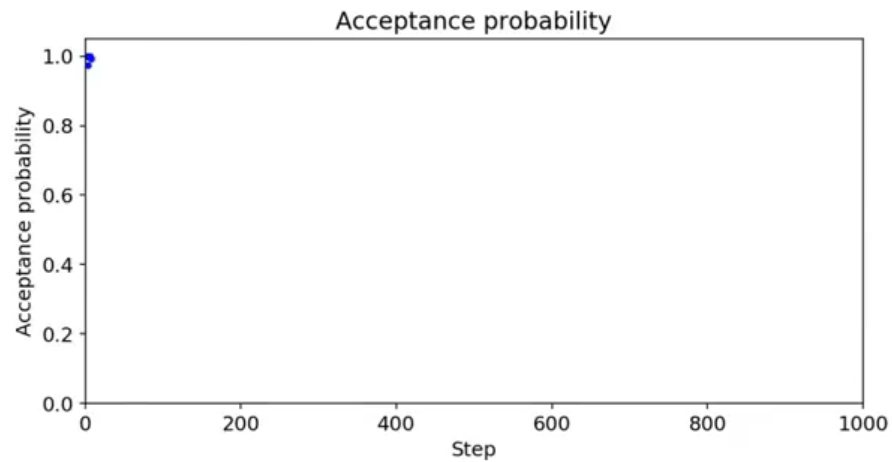
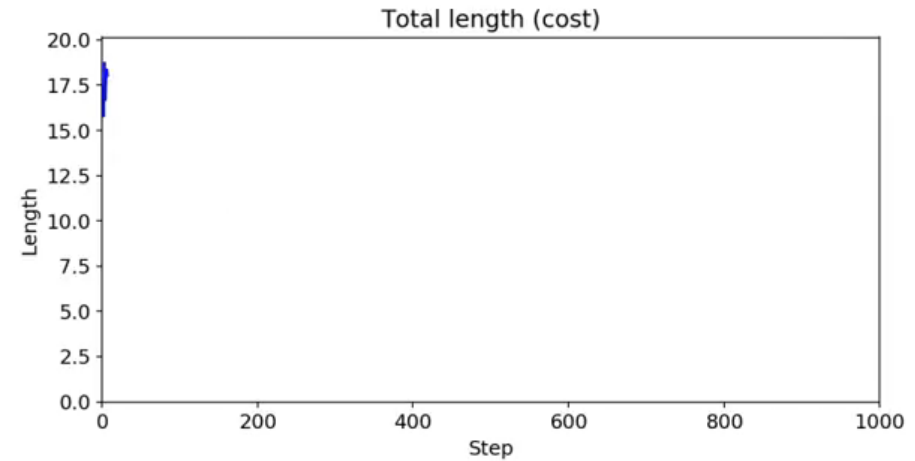
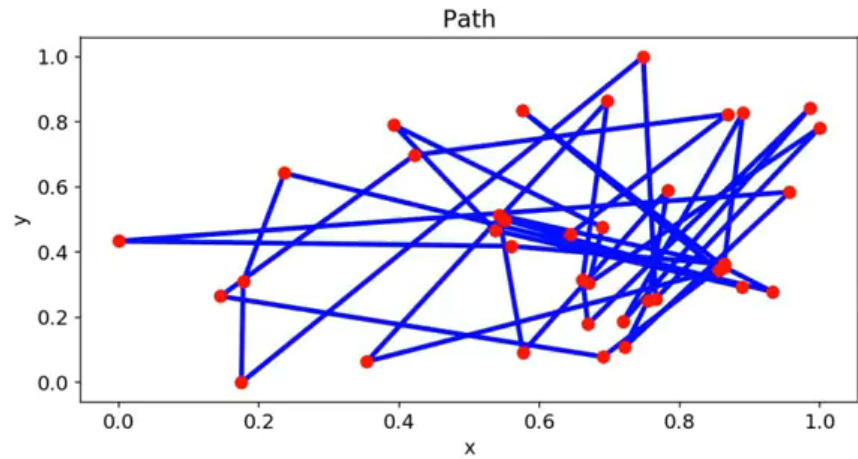
else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

schedule

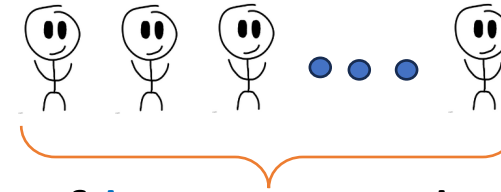
t (time)	T (Temperature)
1	T_MAX
2	T_MAX*0.98
3	T_MAX*(0.98^2)
4	T_MAX*(0.98^3)
...	...
t_MAX	Approach 0



Simulated Annealing on TSP



Local Beam Search



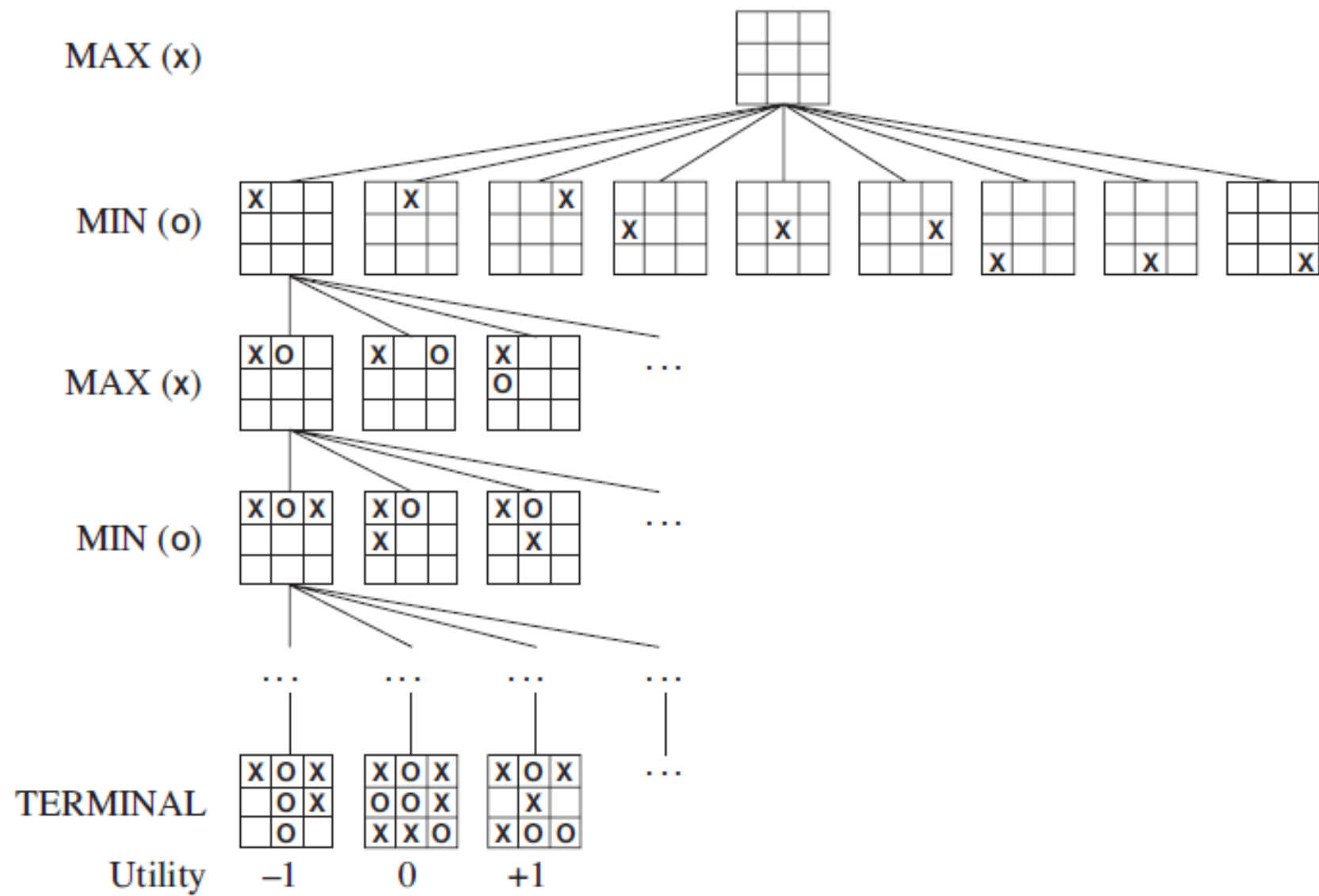
- The **local beam search** algorithm keeps track of **k states** rather than just one.
- It begins with k randomly generated states. At each step, all the successors of all k states are generated.
- If anyone is a goal, the algorithm halts. Otherwise, it **selects the k best successors** from the complete list and repeats.
- However, they quickly become concentrated in a small region of the state space.

MINIMAX Game

- We consider games with two players, whom we call **MAX** and **MIN**.
- MAX moves first, and then they take turns moving until the game is over.
- At the end of the game, points are awarded to the winning player, and penalties are given to the loser.

Example: **Tic-Tac-Toe**

- From the initial state, MAX has nine possible moves. Play alternates between MAX's placing an X and MIN's placing an O.
- The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names).

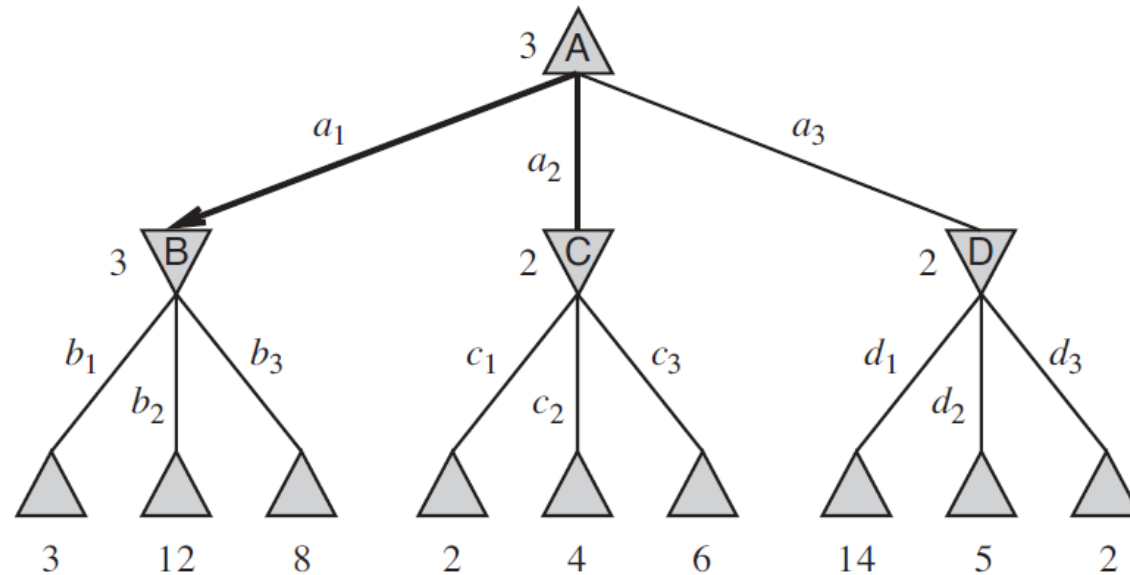


Optimal Decisions in Games

Let's simplify the scenario of the two-ply game tree as follow:

MAX

MIN



- The Δ nodes are “MAX nodes,”.
- The ∇ nodes are “MIN nodes.”

- MAX's best move at the root is a_1 , because it leads to the state with the highest minimax value, and MIN's best reply is b_1 , because it leads to the state with the lowest minimax value.

We can derive the value of **MINIMAX(s)** of each **state s** by applying the following formula.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Example:

$$\begin{aligned} \text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, 4, 6), \min(14, 5, 2)) \\ &= \max(3, 2, 2) \\ &= 3 \end{aligned}$$

Optimal decisions in multiplayer games

- Vector stores utility value of each player.
- All players try to maximize their utility values.

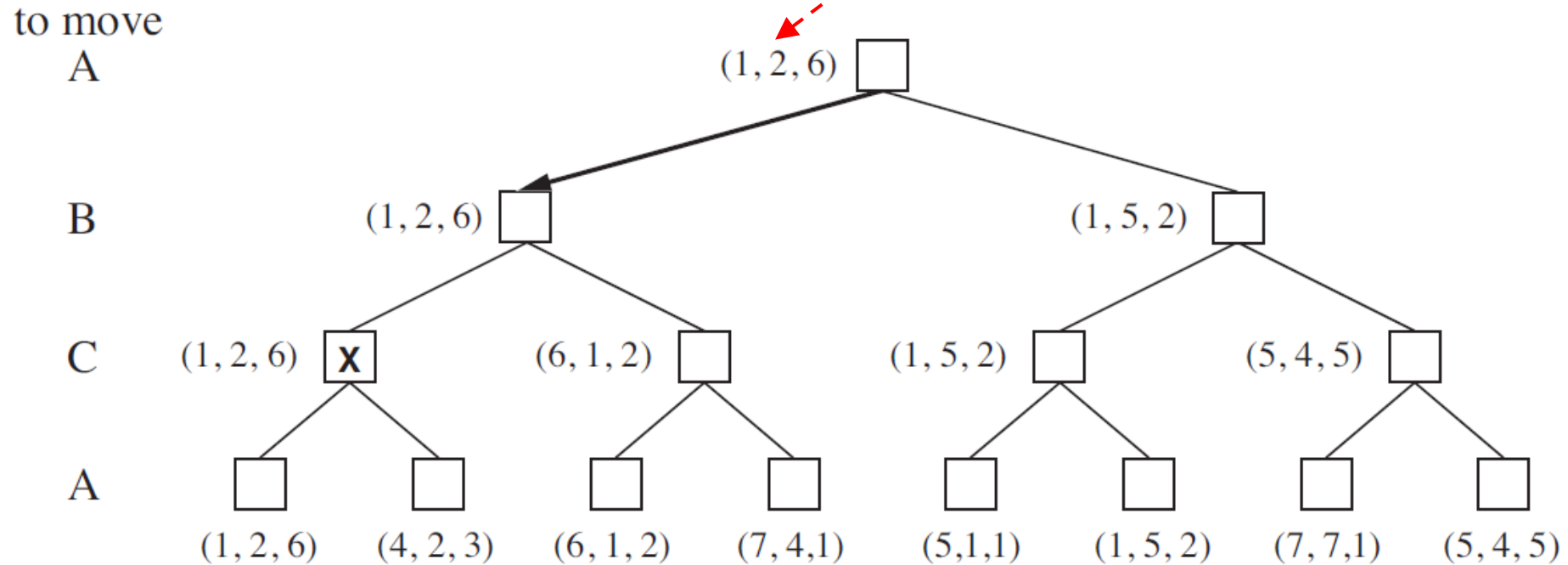
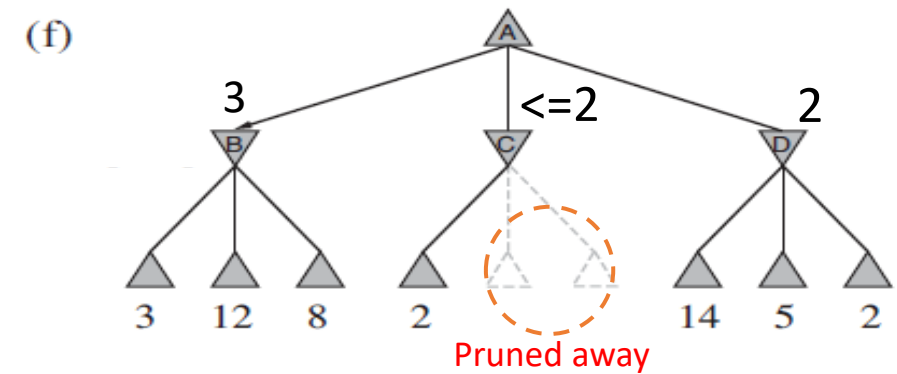
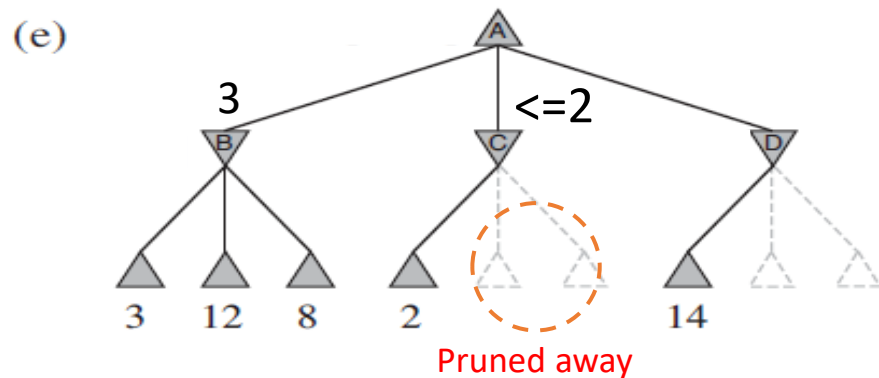
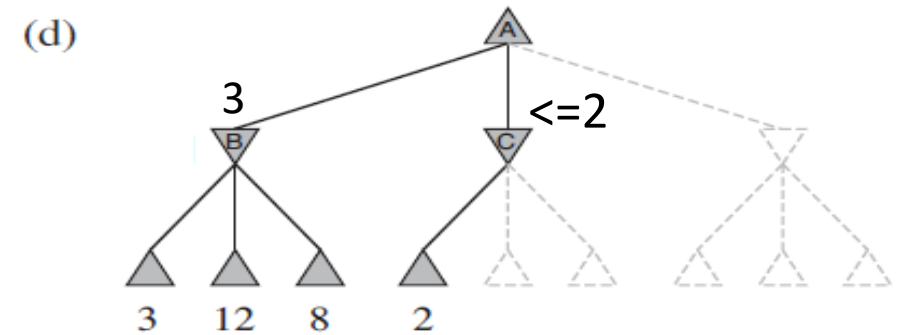
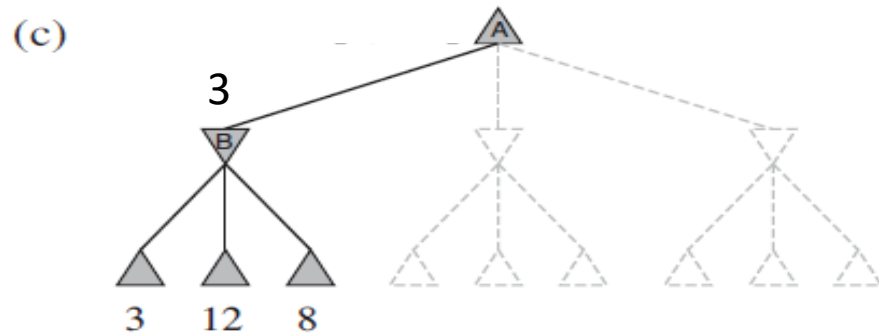
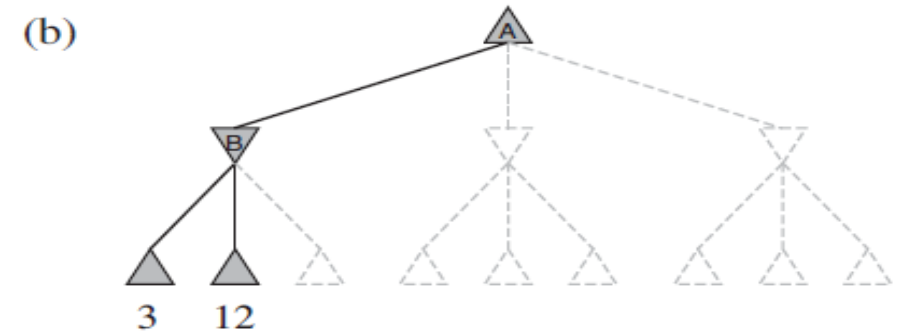
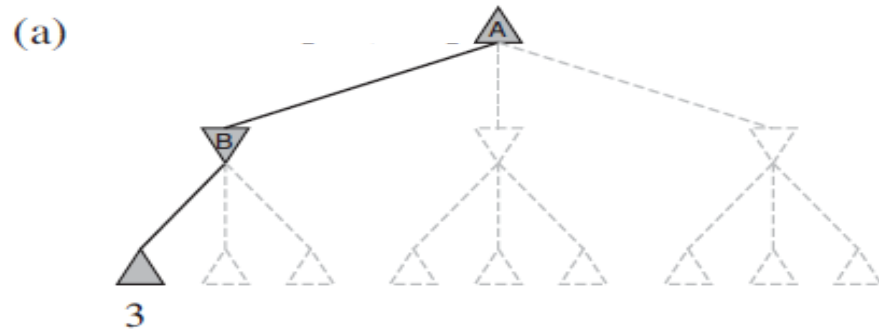


Figure 5.4 The first three plies of a game tree with three players (A , B , C). Each node is labeled with values from the viewpoint of each player. The best move is marked at the root.

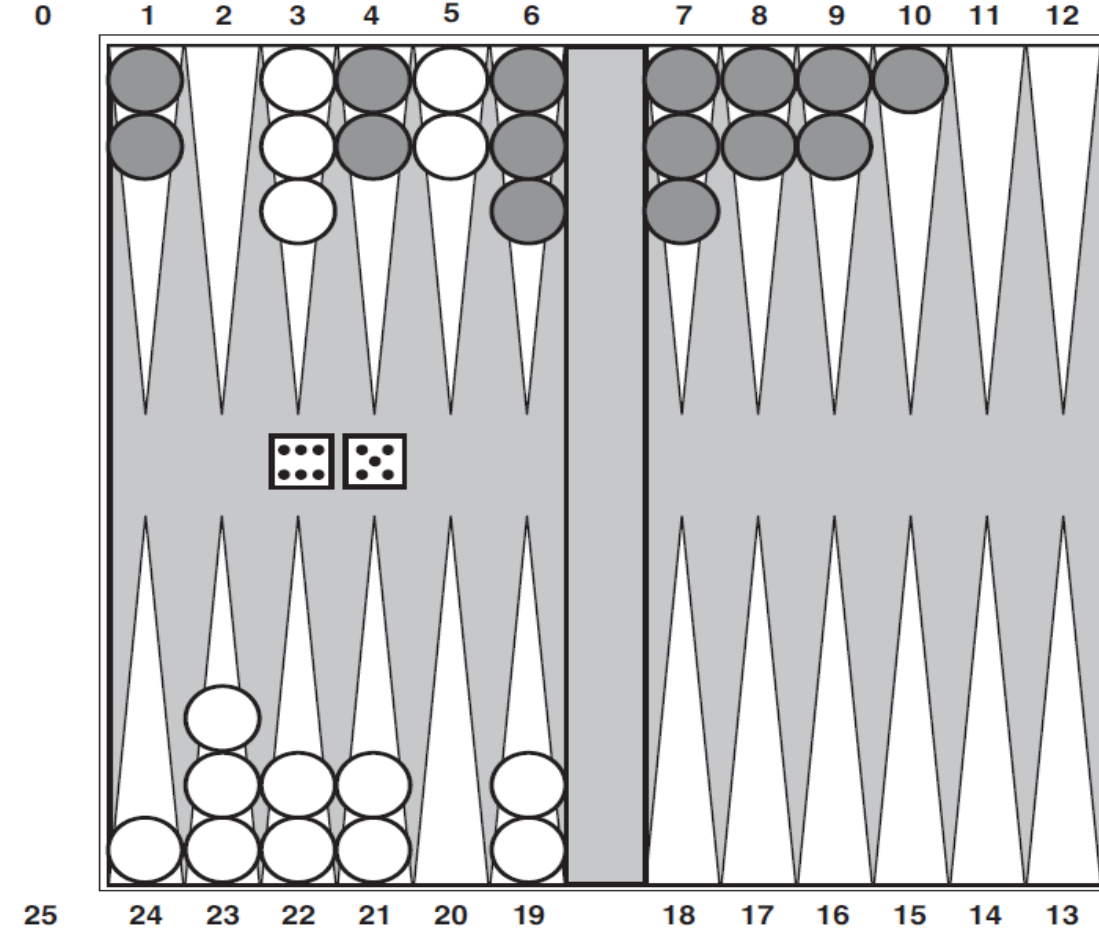
Alpha-Beta Pruning



Stochastic Games

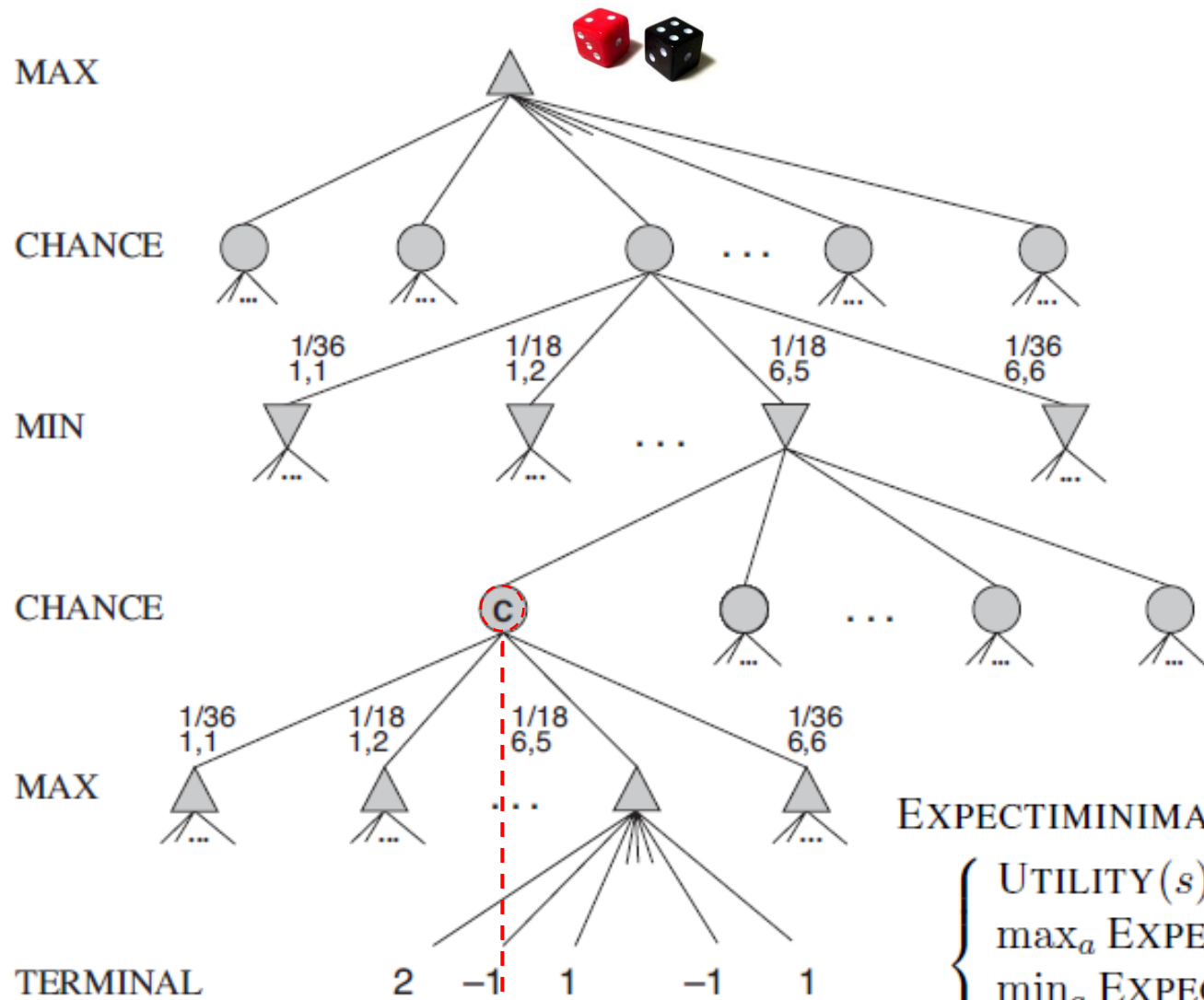


- Backgammon is a typical game that combines luck and skill.
- Dice are rolled at the beginning of a player's turn to determine the legal moves.
- The goal of the game is to move all one's pieces off the board.
- White moves clockwise toward 25, and Black moves counterclockwise toward 0.
- A piece can move to any position unless multiple opponent pieces are there; if there is one opponent, it is captured and must start over.



Backgammon tutorial by Bucky: https://youtu.be/h0D0bQE_Lfc





- Although White knows what his or her own legal moves are, White does not know what Black is going to roll and thus does not know what Black's legal moves will be.
- That means White cannot construct a standard game tree of the sort we saw in chess and tic-tac-toe.
- A CHANCE NODES game tree in backgammon must include **chance nodes** in addition to MAX and MIN nodes.

EXPECTIMINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \\ \sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r)) & \text{if } \text{PLAYER}(s) = \text{CHANCE} \end{cases}$$

Where r represents a possible dice roll (or other chance event) and $\text{RESULT}(s, r)$ is the same state as s , with the additional fact that the result of the dice roll is r .